

ADVANCED HMS

Hospital Management System

Complete Advanced Project Roadmap

Total Phases	5 Phases
Estimated Duration	8–12 Months (Part-time) / 5–7 Months (Full-time)
Core Technology	Django + PostgreSQL + Redis + Celery + React Native
AI Technologies	scikit-learn, Claude API, BioBERT, FHIR, WebSockets
Target Market	South Asian Hospitals (Nepal, India, Bangladesh)
Unique Value	Predictive AI + WhatsApp + Offline + Drug Interaction
Total Apps	20+ Django Apps
Security Level	HIPAA-aligned, Audit Trail, 2FA, Encryption

PROJECT TIMELINE OVERVIEW

Phase	Name	Duration	Focus
Phase 1	Foundation (Done)	2–3 Months	Core HMS — Auth, Appointments, Lab, Payments, Rooms
Phase 2	Production-Grade	1–2 Months	PostgreSQL, Redis, Celery, Security, Audit Trail, 2FA
Phase 3	AI Intelligence	2–3 Months	Drug Check, No-Show ML, Triage, Readmission, Vitals
Phase 4	Integrations	1–2 Months	WhatsApp, FHIR API, Mobile App, ICD-10 Auto Coding
Phase 5	Scale & Deploy	1 Month	Docker, CI/CD, Cloud Deploy, Monitoring, Load Testing

TABLE OF CONTENTS

Phase 1 — Foundation — Core HMS (Already Built)

Phase 2 — Production-Grade Infrastructure

- 2.1 — PostgreSQL Migration
- 2.2 — Redis + Celery Async Tasks
- 2.3 — Complete Audit Trail System
- 2.4 — Two-Factor Authentication
- 2.5 — Field-Level Encryption
- 2.6 — Rate Limiting & Brute Force Protection
- 2.7 — Role-Based Field Security

Phase 3 — AI & Machine Learning Intelligence

- 3.1 — Drug Interaction Checker
- 3.2 — Appointment No-Show Predictor
- 3.3 — AI Triage System
- 3.4 — Readmission Risk Predictor
- 3.5 — Vital Signs Anomaly Detection
- 3.6 — AI Medical Report Reader
- 3.7 — Heart Attack Risk Predictor
- 3.8 — Smart Medicine Inventory Predictor

Phase 4 — Advanced Integrations

- 4.1 — WhatsApp Integration (Twilio)
- 4.2 — HL7 FHIR API
- 4.3 — ICD-10 Auto Coding
- 4.4 — React Native Mobile App
- 4.5 — Telemedicine (Video Calls)

Phase 5 — Scale, Deploy & Monitor

- 5.1 — Docker + Docker Compose
- 5.2 — CI/CD Pipeline
- 5.3 — Cloud Deployment
- 5.4 — Monitoring & Alerting

Appendix A — Complete Technology Stack

Appendix B — All Database Models

Appendix C — Security Checklist

Appendix D — What Makes This Unique

PHASE 1

FOUNDATION — Core HMS

Duration: 2–3 Months | Status: COMPLETE ■

Phase 1 is complete! You have built the foundational HMS. This section summarizes what was built so you have a clear reference before moving to Phase 2.

App	Models	Key Features
users	User, PatientProfile	5 roles, email auth, email activation, profile management
clinical	Specialization, Doctor, Shift, DoctorSchedule, DoctorLeave	Schedule management, doctor schedules, leaves, search
appointments	Appointment, WaitingList	4-step booking, slot generation, QR code, PDF, cancel
prescriptions	Prescription, PrescriptionItem	Write prescriptions, multiple medicines, PDF download
payments	Payment, Refund	Auto-create, 3 payment types, receipts, refunds, revenue
lab	TestTemplate, TestField, TestSchedule, TestResult, TestResultLow	Simple booking, test results, auto bill, PDF
rooms	Room, Admission	4 room types, admit/discharge, auto bill, discharge PDF
notifications	Notification	In-app + email, 15 event types, live bell count

PHASE 2

PRODUCTION-GRADE INFRASTRUCTURE

Duration: 1–2 Months | Priority: HIGH — Do Before Phase 3

Do NOT skip Phase 2. Adding AI on top of weak infrastructure creates an unreliable system. Phase 2 makes your app production-ready, scalable, and secure.

2.1 PostgreSQL Migration

SQLite cannot handle concurrent users, is not suitable for production, and has limited query capabilities. PostgreSQL is the industry standard for healthcare systems.

Why PostgreSQL?

- Handles 1000+ concurrent users without data corruption
- Full ACID compliance — no data loss in crashes
- Advanced indexing — fast patient search on large datasets
- JSON fields for flexible medical data storage
- Full-text search built-in — search patient notes
- Required for HIPAA-compliant audit logging

Installation & Migration

```
# Install
pip install psycopg2-binary dj-database-url

# settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.getenv('DB_NAME', 'hmsdb'),
        'USER': os.getenv('DB_USER', 'hmsuser'),
        'PASSWORD': os.getenv('DB_PASSWORD'),
        'HOST': os.getenv('DB_HOST', 'localhost'),
        'PORT': os.getenv('DB_PORT', '5432'),
        'CONN_MAX_AGE': 60, # Connection pooling
    }
}
```

New Models to Add for PostgreSQL Features

```
# Use PostgreSQL-specific fields for better performance
from django.contrib.postgres.fields import ArrayField
from django.contrib.postgres.search import SearchVectorField

class PatientProfile(models.Model):
    # Full-text search on patient name/notes
    search_vector = SearchVectorField(null=True, blank=True)
    # Store multiple allergies as array
    allergies = ArrayField(
        models.CharField(max_length=100), blank=True, default=list
    )
    # Store blood type history as JSON
    medical_history = models.JSONField(default=dict, blank=True)
```

2.2 Redis + Celery — Async Task Queue

Currently all emails, PDFs, and notifications happen synchronously — making the user wait. With Celery, these happen in the background instantly returning response to user.

Task	Current (Sync)	With Celery (Async)
Send email notification	User waits 3 seconds	Instant — background
Generate PDF receipt	User waits 2 seconds	Instant — background
Generate lab report PDF	User waits 4 seconds	Instant — background
Send WhatsApp message	Blocks request	Background queue
Run ML prediction	Blocks for 500ms	Background — result via WebSocket
Appointment reminders	Manual or cron	Celery Beat scheduled task
Daily report generation	Not possible	Scheduled every night

```
pip install celery redis django-celery-beat

# settings.py
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'
CELERY_ACCEPT_CONTENT = ['json']

# hms_project/celery.py
import os
from celery import Celery
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'hms_project.settings')
app = Celery('hms_project')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()

# Example task – notifications/tasks.py
from celery import shared_task

@shared_task
def send_appointment_email_task(appointment_id):
    from appointments.models import Appointment
    from notifications.utils import notify_appointment_booked
    apt = Appointment.objects.get(pk=appointment_id)
    notify_appointment_booked(apt)

# In view – instead of calling directly:
send_appointment_email_task.delay(appointment.id) # Non-blocking!
```

Celery Beat — Scheduled Tasks

Schedule	Task	Purpose
Every day 9 AM	send_appointment_reminders	Remind patients of tomorrow's appointments
Every day 8 AM	check_no_show_predictions	Run ML on today's appointments
Every hour	check_critical_vitals	Alert if patient vitals are abnormal
Every Monday 7 AM	generate_weekly_report	Admin weekly revenue report email
Every night 11 PM	backup_database	Auto DB backup to cloud storage
Every 15 minutes	check_medicine_expiry	Alert for expiring medicines

2.3 Complete Audit Trail System

Every single action in the hospital must be logged. This is required for HIPAA compliance, legal protection, and detecting unauthorized access.

```
# audit/models.py
class AuditLog(models.Model):
    ACTION_CHOICES = [
        ('CREATE', 'Created'), ('VIEW', 'Viewed'),
        ('UPDATE', 'Updated'), ('DELETE', 'Deleted'),
        ('LOGIN', 'Logged In'), ('LOGOUT', 'Logged Out'),
        ('DOWNLOAD', 'Downloaded'), ('FAILED_LOGIN', 'Failed Login'),
    ]
    user = models.ForeignKey('users.User', null=True, on_delete=models.SET_NULL)
    action = models.CharField(max_length=15, choices=ACTION_CHOICES)
    model_name = models.CharField(max_length=100) # 'Prescription'
    object_id = models.IntegerField(null=True)
    object_repr = models.CharField(max_length=200) # 'Rx #5 for Prabin'
    old_values = models.JSONField(null=True) # Before change
    new_values = models.JSONField(null=True) # After change
    ip_address = models.GenericIPAddressField(null=True)
    user_agent = models.TextField(blank=True)
    timestamp = models.DateTimeField(auto_now_add=True)
    is_sensitive = models.BooleanField(default=False)

    class Meta:
        # Nobody can delete audit logs – not even superuser
        # Override delete() to raise PermissionDenied
        ordering = ['-timestamp']
        indexes = [models.Index(fields=['user', 'timestamp'])]
```

2.4 Two-Factor Authentication (2FA)

Doctors and Admins handle sensitive data. 2FA prevents unauthorized access even if password is stolen — critical for hospital security.

```
pip install django-otp qrcode

# Who requires 2FA:
ROLES_REQUIRING_2FA = ['DOCTOR', 'ADMIN', 'LAB_TECHNICIAN']

# 2FA Methods supported:
# 1. TOTP (Google Authenticator / Authy)
# 2. Email OTP (6-digit code to email)
# 3. SMS OTP (via Twilio)

# users/models.py – Add to User model:
two_factor_enabled = models.BooleanField(default=False)
two_factor_method = models.CharField(
    max_length=10,
    choices=[('TOTP', 'Authenticator'), ('EMAIL', 'Email'), ('SMS', 'SMS')],
    default='EMAIL'
)
```

2.5 Field-Level Data Encryption

Sensitive patient data (diagnosis, prescriptions, test results) must be encrypted in the database. Even if database is hacked, data is unreadable.

```
pip install django-encrypted-model-fields

# Which fields to encrypt:
from encrypted_model_fields.fields import EncryptedTextField

class Prescription(models.Model):
    notes = EncryptedTextField()      # Diagnosis encrypted
    advice = EncryptedTextField()     # Advice encrypted

class TestResultItem(models.Model):
    value = EncryptedTextField()      # Lab values encrypted

class Admission(models.Model):
    reason = EncryptedTextField()      # Admission reason encrypted
    discharge_notes = EncryptedTextField()

# Add to settings.py:
FIELD_ENCRYPTION_KEY = os.getenv('FIELD_ENCRYPTION_KEY')
```

2.6 Rate Limiting & Security

```
pip install django-ratelimit django-axes

# Block after 5 failed login attempts
AXES_FAILURE_LIMIT = 5
AXES_COOLOFF_TIME = 1 # Hours
AXES_LOCKOUT_TEMPLATE = 'users/locked_out.html'

# Rate limit API views
from django_ratelimit.decorators import ratelimit

@ratelimit(key='ip', rate='10/m', block=True) # 10 requests/minute
def api_appointment_create(request):
    ...
```

2.7 Role-Based Field Level Security

Field	Patient	Doctor	Receptionist	Lab Tech	Admin
Patient Name	Own only	■ Yes	■ Yes	■ Yes	■ Yes
Diagnosis	Own only	■ Yes	■ No	■ No	■ Yes
Prescription	Own only	Own only	■ No	■ No	■ Yes
Lab Values	Own only	■ Yes	■ No	■ Yes	■ Yes
Payment Amount	Own only	■ No	■ Yes	■ No	■ Yes
Admin Audit Logs	■ No	■ No	■ No	■ No	■ Yes

PHASE 3

AI & MACHINE LEARNING INTELLIGENCE

Duration: 2–3 Months | This is what makes your HMS UNIQUE

Phase 3 transforms your HMS from a record-keeping system into an intelligent clinical decision support system. This is what NO other HMS in South Asia has.

3.1 Drug Interaction Checker

The most critical AI feature. When a doctor prescribes medicines, the system automatically checks for dangerous interactions with the patient's current medications, allergies, and medical conditions. Prevents medication errors — the 3rd leading cause of hospital deaths.

How It Works

- Doctor starts writing a prescription
- System loads patient's current medicines from all previous prescriptions
- For each new medicine added, system checks against existing medicines
- Checks against patient's known allergies
- Checks dose appropriateness for patient age/weight/kidney function
- Shows WARNING (yellow) or DANGER (red) alert immediately
- Doctor must acknowledge warning before saving
- All warnings logged in audit trail

Models

```
# drug_checker/models.py

class DrugInteraction(models.Model):
    SEVERITY_CHOICES = [
        ('MINOR', 'Minor'), ('MODERATE', 'Moderate'),
        ('MAJOR', 'Major'), ('CONTRAINDICATED', 'Contraindicated'),
    ]
    drug_a = models.CharField(max_length=200) # Generic name
    drug_b = models.CharField(max_length=200)
    severity = models.CharField(max_length=20, choices=SEVERITY_CHOICES)
    description = models.TextField() # Clinical explanation
    mechanism = models.TextField() # Why it happens
    management = models.TextField() # What doctor should do
    source = models.CharField(max_length=100) # FDA, WHO etc.

class DrugAllergyAlert(models.Model):
    patient = models.ForeignKey('users.PatientProfile', on_delete=models.CASCADE)
    drug_name = models.CharField(max_length=200)
    reaction = models.TextField() # What happens on exposure
    severity = models.CharField(max_length=20)
    recorded_by = models.ForeignKey('users.User', on_delete=models.SET_NULL, null=True)
    recorded_at = models.DateTimeField(auto_now_add=True)

class PrescriptionInteractionLog(models.Model):
    prescription = models.ForeignKey('prescriptions.Prescription',
                                     on_delete=models.CASCADE)
    drug_a = models.CharField(max_length=200)
```

```
drug_b = models.CharField(max_length=200)
severity = models.CharField(max_length=20)
doctor_acknowledged = models.BooleanField(default=False)
acknowledged_at = models.DateTimeField(null=True)
```

Technology Stack

Component	Technology	Details
Interaction Database	DrugBank API / RxNorm	200,000+ drug interaction pairs — free tier available
Fuzzy Drug Matching	RapidFuzz library	Matches 'Amoxicillin 500mg' to 'amoxicillin' in database
API Check	OpenFDA Drug API	Free FDA drug database — real-time checks
Local DB	PostgreSQL	Cache interactions locally for speed
Real-time UI	JavaScript + HTMX	Show warning without page reload as doctor types

Install

```
pip install rapidfuzz requests

# Import FDA drug interaction data (free):
# https://open.fda.gov/apis/drug/drugsfda/
# Run once to populate DrugInteraction table:
python manage.py import_drug_interactions

# Real-time check API endpoint:
# GET /api/drug-check/?drug_a=amoxicillin&drug_b=warfarin&patient_id=5
# Returns: {severity: 'MAJOR', description: '...', management: '...'}
```

3.2 Appointment No-Show Predictor

In South Asian hospitals, 35-45% of booked appointments result in no-shows. This wastes doctor time and revenue. An ML model predicts which patients are likely to not show up, enabling proactive intervention.

ML Features (What the model learns from)

Feature Category	Features Used	Impact
Patient History	Previous no-show rate, cancellation count, rescheduled count	HIGH
Appointment Details	Day of week, time of day, days until appointment, clinic appointment type	MEDIUM
Patient Demographics	Age, gender, distance from hospital (city/district)	MEDIUM
External Factors	Day before public holiday, month (festival season)	MEDIUM
Booking Behavior	How far in advance booked, how many times rescheduled	HIGH
Communication	Has phone number, email, previous response to reminders	MEDIUM
Financial	Payment status of previous appointments	HIGH

Models

```
# ml_noshow/models.py

class NoShowPrediction(models.Model):
    appointment = models.OneToOneField(
        'appointments.Appointment', on_delete=models.CASCADE)
    probability = models.DecimalField(max_digits=5, decimal_places=2)
    risk_level = models.CharField(
        max_length=10,
```

```

        choices=[('LOW', 'Low'), ('MEDIUM', 'Medium'), ('HIGH', 'High')]
    )
    features_used = models.JSONField() # Store what features were used
    action_taken = models.CharField(
        max_length=20,
        choices=[
            ('NONE', 'No Action'),
            ('SMS_SENT', 'SMS Sent'),
            ('WHATSAPP_SENT', 'WhatsApp Sent'),
            ('DOUBLE_BOOKED', 'Slot Double Booked'),
        ],
        default='NONE'
    )
    actual_outcome = models.CharField(
        max_length=10,
        choices=[('SHOWED', 'Showed'), ('NOSHOW', 'No Show'), ('PENDING', 'Pending')],
        default='PENDING'
    )
    predicted_at = models.DateTimeField(auto_now_add=True)

class NoShowMLModel(models.Model):
    """Tracks ML model versions and performance"""
    version = models.CharField(max_length=20)
    algorithm = models.CharField(max_length=50) # 'XGBoost v1.2'
    accuracy = models.DecimalField(max_digits=5, decimal_places=2)
    precision = models.DecimalField(max_digits=5, decimal_places=2)
    recall = models.DecimalField(max_digits=5, decimal_places=2)
    f1_score = models.DecimalField(max_digits=5, decimal_places=2)
    trained_on = models.DateTimeField()
    is_active = models.BooleanField(default=False)
    model_file = models.FileField(upload_to='ml_models/')

```

Training Script

```

# ml_noshow/train.py
import pandas as pd
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import joblib

# Build dataset from your appointment history
def build_dataset():
    from appointments.models import Appointment
    apts = Appointment.objects.all().values(
        'id', 'date', 'start_time', 'status',
        'patient__id', 'doctor__id', 'created_at'
    )
    # Feature engineering...
    # Target: 1 = no show, 0 = showed up

model = XGBClassifier(n_estimators=100, max_depth=6)
model.fit(X_train, y_train)
joblib.dump(model, 'ml_noshow/noshow_model.pkl')

# Celery task – runs after each appointment is booked:

```

```
@shared_task
def predict_noshow(appointment_id):
    model = joblib.load('ml_noshow/noshow_model.pkl')
    # ... extract features, predict
    if prob > 0.7: # HIGH risk
        send_whatsapp_reminder.delay(appointment_id)
        # Double book the slot as backup
```

3.3 AI Triage System

When a patient arrives or registers, they fill a quick symptom questionnaire. The AI assigns a triage priority level, ensuring critical patients are seen immediately while reducing burden on doctors for minor cases.

Triage Priority Levels

Level	Color	Response Time	Example Conditions
P1 — CRITICAL	■ Red	Immediate	Chest pain, difficulty breathing, stroke symptoms, severe bleeding
P2 — URGENT	■ Orange	< 30 minutes	High fever (>103°F), severe pain, head injury, abdominal pain
P3 — SEMI-URGENT	■ Yellow	< 2 hours	Moderate pain, persistent vomiting, minor injuries
P4 — NON-URGENT	■ Green	< 4 hours	Cold, minor cuts, follow-up visits, mild symptoms

Models

```
# triage/models.py

class TriageAssessment(models.Model):
    patient = models.ForeignKey('users.PatientProfile',
                               on_delete=models.CASCADE)
    priority = models.CharField(
        max_length=12,
        choices=[('P1_CRITICAL', 'Critical'), ('P2_URGENT', 'Urgent'),
                 ('P3_SEMI', 'Semi-Urgent'), ('P4_NON', 'Non-Urgent')]
    )
    chief_complaint = models.TextField() # Main symptom in patient's words
    symptoms = models.JSONField() # List of selected symptoms
    vital_signs = models.JSONField(null=True) # BP, pulse, temp, SpO2
    pain_scale = models.IntegerField(null=True) # 0-10
    ai_reasoning = models.TextField() # Why AI chose this priority
    ai_confidence = models.DecimalField(max_digits=5, decimal_places=2)
    nurse_override = models.CharField(max_length=12, null=True, blank=True)
    nurse_notes = models.TextField(blank=True)
    assessed_at = models.DateTimeField(auto_now_add=True)
    seen_by_doctor_at = models.DateTimeField(null=True)
    wait_time_minutes = models.IntegerField(null=True) # Actual wait time

class SymptomDatabase(models.Model):
    name = models.CharField(max_length=200)
    keywords = models.JSONField() # ['chest pain', 'chest tightness']
    urgency_weight = models.DecimalField(max_digits=4, decimal_places=2)
    body_system = models.CharField(max_length=50) # Cardiac, Respiratory...
    red_flag = models.BooleanField(default=False) # Always = P1 if True
```

Technology

```
# Use Claude API for nuanced triage:
import anthropic
```

```

TRIAGE_PROMPT = '''
You are an emergency triage nurse AI for HMS Hospital.
Analyze the following patient symptoms and assign a priority level.

Patient info: {age} year old {gender}
Chief complaint: {complaint}
Symptoms selected: {symptoms}
Vital signs: {vitals}
Pain scale: {pain}/10

Respond in JSON format:
{{
  "priority": "P1_CRITICAL|P2_URGENT|P3_SEMI|P4_NON",
  "reasoning": "brief clinical reasoning",
  "red_flags": ["list of concerning signs"],
  "immediate_actions": ["what staff should do now"],
  "confidence": 0.95
}}
'''

```

3.4 Readmission Risk Predictor

Predicts whether a patient will be readmitted within 30 days of discharge. High-risk patients get automatic follow-up scheduling and daily check-in messages. Reduces costly readmissions.

```

# ml_readmission/models.py

class ReadmissionRisk(models.Model):
    admission = models.OneToOneField('rooms.Admission', on_delete=models.CASCADE)
    risk_score = models.DecimalField(max_digits=5, decimal_places=2) # 0-100
    risk_level = models.CharField(max_length=6,
        choices=[('LOW', 'Low'), ('MEDIUM', 'Medium'), ('HIGH', 'High')])
    risk_factors = models.JSONField() # Why high risk
    recommended_followup_days = models.IntegerField(default=14)
    followup_scheduled = models.BooleanField(default=False)
    actual_readmitted = models.BooleanField(null=True) # Ground truth
    predicted_at = models.DateTimeField(auto_now_add=True)

# Features used for prediction:
# - Primary diagnosis (ICD code)
# - Number of previous admissions
# - Length of current stay
# - Age and comorbidities (diabetes, hypertension)
# - Lab values at discharge (abnormal = higher risk)
# - Whether patient lives alone
# - Days since last admission

```

3.5 Vital Signs Anomaly Detection

For admitted patients, nurses enter vitals every few hours. ML detects deterioration patterns BEFORE the patient becomes critical. Catches sepsis, cardiac events, and respiratory failure earlier.

```
# vitals/models.py

class VitalSigns(models.Model):
    admission = models.ForeignKey('rooms.Admission',
                                  on_delete=models.CASCADE,
                                  related_name='vitals')
    recorded_by = models.ForeignKey('users.User', on_delete=models.SET_NULL, null=True)
    temperature = models.DecimalField(max_digits=5, decimal_places=2, null=True) # Celsius
    systolic_bp = models.IntegerField(null=True) # mmHg
    diastolic_bp = models.IntegerField(null=True)
    heart_rate = models.IntegerField(null=True) # bpm
    respiratory_rate = models.IntegerField(null=True) # breaths/min
    oxygen_saturation = models.DecimalField(max_digits=5, decimal_places=2, null=True) # %
    blood_glucose = models.DecimalField(max_digits=6, decimal_places=2, null=True)
    consciousness = models.CharField(
        max_length=10,
        choices=[('A', 'Alert'), ('V', 'Voice'), ('P', 'Pain'), ('U', 'Unresponsive')],
        default='A'
    )
    mews_score = models.IntegerField(null=True) # Auto-calculated
    recorded_at = models.DateTimeField(auto_now_add=True)

class VitalAlert(models.Model):
    admission = models.ForeignKey('rooms.Admission', on_delete=models.CASCADE)
    alert_type = models.CharField(max_length=30) # 'SEPSIS_RISK', 'CARDIAC_EVENT'
    severity = models.CharField(max_length=10) # 'WARNING', 'CRITICAL'
    message = models.TextField()
    mews_score = models.IntegerField()
    notified_doctor = models.BooleanField(default=False)
    resolved = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
```

MEWS Score Thresholds

MEWS Score	Risk Level	Action Required
0–2	LOW — Normal	Routine monitoring every 4-6 hours
3–4	MEDIUM — Watch	Increase monitoring to every 2 hours, notify nurse
5–6	HIGH — Concern	Notify doctor immediately, hourly monitoring
7+	CRITICAL	Emergency response, consider ICU transfer

3.6 AI Medical Report Reader

Patient uploads any hospital report (PDF, image). Claude AI reads it, explains findings in simple language, identifies abnormal values, and suggests follow-up actions.

```
# ai_reader/models.py

class ReportAnalysis(models.Model):
    REPORT_TYPES = [
        ('LAB', 'Lab Report'), ('XRAY', 'X-Ray'),
        ('MRI', 'MRI'), ('CT', 'CT Scan'),
        ('ECG', 'ECG'), ('ECHO', 'Echocardiogram'),
        ('OTHER', 'Other'),
    ]
    user = models.ForeignKey('users.User', on_delete=models.CASCADE)
    original_file = models.FileField(upload_to='ai_reports/')
    report_type = models.CharField(max_length=10, choices=REPORT_TYPES)
    ai_summary = models.TextField() # Plain English summary
    abnormal_findings = models.JSONField() # List of abnormal values
    recommendations = models.TextField() # Suggested actions
    urgency_flag = models.CharField(
        max_length=10,
        choices=[('NORMAL', 'Normal'), ('WATCH', 'Watch'), ('URGENT', 'Urgent')]
    )
    tokens_used = models.IntegerField(default=0) # Track API cost
    created_at = models.DateTimeField(auto_now_add=True)

# Technology: Anthropic Claude claude-opus-4-6 (supports PDF + images)
# pip install anthropic
```

3.7 Heart Attack Risk Predictor

```
# ml_heart/models.py

class HeartRiskAssessment(models.Model):
    patient = models.ForeignKey('users.PatientProfile', on_delete=models.CASCADE)
    # Input features (UCI Heart Disease dataset)
    age = models.IntegerField()
    sex = models.IntegerField() # 0=Female, 1=Male
    chest_pain_type = models.IntegerField() # 0-3
    resting_bp = models.IntegerField() # mmHg
    cholesterol = models.IntegerField() # mg/dl
    fasting_blood_sugar = models.IntegerField() # 0 or 1
    resting_ecg = models.IntegerField() # 0-2
    max_heart_rate = models.IntegerField()
    exercise_angina = models.IntegerField() # 0 or 1
    st_depression = models.DecimalField(max_digits=4, decimal_places=1)
    st_slope = models.IntegerField() # 0-2
    major_vessels = models.IntegerField() # 0-3
    thalassemia = models.IntegerField() # 1-3
    # Output
    risk_score = models.DecimalField(max_digits=5, decimal_places=2) # %
    risk_level = models.CharField(max_length=8,
        choices=[('LOW', 'Low'), ('MEDIUM', 'Medium'), ('HIGH', 'High')])
    model_version = models.CharField(max_length=20)
    assessed_at = models.DateTimeField(auto_now_add=True)
```

```

# Link to appointment if done during visit
appointment = models.ForeignKey(
    'appointments.Appointment', null=True, blank=True,
    on_delete=models.SET_NULL
)

# Algorithm: Random Forest + XGBoost ensemble
# Dataset: UCI Heart Disease (Cleveland) – 303 samples
# pip install scikit-learn xgboost joblib

```

3.8 Smart Medicine Inventory Predictor

```

# inventory/models.py

class Medicine(models.Model):
    name = models.CharField(max_length=200)
    generic_name = models.CharField(max_length=200)
    category = models.CharField(max_length=100)
    unit = models.CharField(max_length=20) # tablets, vials, bottles
    current_stock = models.IntegerField()
    reorder_level = models.IntegerField() # Alert when stock falls below
    unit_cost = models.DecimalField(max_digits=8, decimal_places=2)
    supplier = models.CharField(max_length=200)
    expiry_date = models.DateField(null=True)

class MedicineUsage(models.Model):
    medicine = models.ForeignKey(Medicine, on_delete=models.CASCADE)
    quantity_used = models.IntegerField()
    used_date = models.DateField()
    prescription = models.ForeignKey(
        'prescriptions.PrescriptionItem', null=True,
        on_delete=models.SET_NULL
    )

class StockPrediction(models.Model):
    medicine = models.ForeignKey(Medicine, on_delete=models.CASCADE)
    predicted_usage_7d = models.IntegerField() # Next 7 days
    predicted_usage_30d = models.IntegerField() # Next 30 days
    recommended_order = models.IntegerField()
    predicted_stockout_date = models.DateField(null=True)
    predicted_at = models.DateField(auto_now_add=True)

# Algorithm: Facebook Prophet / ARIMA for time-series
# pip install prophet

```

4.1 WhatsApp Integration (Twilio)

80% of patients in Nepal and South Asia use WhatsApp daily. Email notifications are often ignored. WhatsApp messages have 98% open rate vs 20% for email.

Trigger	WhatsApp Message Sent
Appointment Booked	Your appointment with Dr. {name} is confirmed for {date} at {time}. Reply CANCEL to cancel.
24h Before Appointment	Reminder: You have appointment tomorrow at {time}. Reply CONFIRM or CANCEL.
No-Show Risk High	We noticed you haven't confirmed. Your appointment is in 2 hours. Reply YES to confirm.
Prescription Ready	Your prescription from Dr. {name} is ready. Login to HMS to download: {link}
Lab Report Ready	Your {test_name} report is ready. Download here: {link}
Payment Due	You have an outstanding payment of Rs. {amount}. Pay here: {link}
Discharge	You have been discharged. Total bill: Rs. {amount}. Download summary: {link}
Follow Up Reminder	Time for your follow-up! Book appointment: {link}

```

pip install twilio

# settings.py
TWILIO_ACCOUNT_SID = os.getenv('TWILIO_ACCOUNT_SID')
TWILIO_AUTH_TOKEN = os.getenv('TWILIO_AUTH_TOKEN')
TWILIO_WHATSAPP_NUMBER = 'whatsapp:+14155238886' # Twilio sandbox

# notifications/whatsapp.py
from twilio.rest import Client

def send_whatsapp(patient_phone, message):
    client = Client(settings.TWILIO_ACCOUNT_SID, settings.TWILIO_AUTH_TOKEN)
    client.messages.create(
        from_=settings.TWILIO_WHATSAPP_NUMBER,
        body=message,
        to=f'whatsapp:+977{patient_phone}' # Nepal code
    )

# WhatsApp Bot – patient can reply to check status:
# 'STATUS' -> returns current appointment status
# 'CANCEL' -> cancels upcoming appointment
# 'REPORTS' -> sends link to view reports

```

4.2 HL7 FHIR API

FHIR (Fast Healthcare Interoperability Resources) is the international standard for health data exchange. Adding FHIR makes your HMS compatible with government health systems, insurance companies, and other hospitals globally.

```

pip install fhirclient

# FHIR endpoints your HMS exposes:
# GET /fhir/Patient/{id} -> Patient demographics

```

```
# GET /fhir/Appointment/{id}      -> Appointment details
# GET /fhir/Observation/{id}      -> Lab results
# GET /fhir/MedicationRequest/{id} -> Prescriptions
# GET /fhir/Encounter/{id}        -> Hospital admission

# Why this matters:
# - Insurance companies can pull patient data automatically
# - Government health ministry can access population data
# - Patient can take their records to any FHIR-compatible hospital
# - Required for government hospital tenders in many countries
```

4.3 ICD-10 Auto Coding

ICD-10 codes are required for insurance claims and government reporting. Currently doctors waste 30-60 minutes per day on manual coding. AI maps diagnosis text to ICD codes automatically.

```
# icd_coding/models.py

class ICD10Code(models.Model):
    code = models.CharField(max_length=10)    # 'I21.0'
    description = models.CharField(max_length=500) # 'Acute MI of anterior wall'
    category = models.CharField(max_length=100)
    parent_code = models.CharField(max_length=10, null=True)

class DiagnosisCoding(models.Model):
    prescription = models.ForeignKey(
        'prescriptions.Prescription', on_delete=models.CASCADE)
    raw_text = models.TextField()             # Doctor's text: 'chest pain, angina'
    suggested_codes = models.JSONField()      # Top 3 ICD matches
    selected_code = models.ForeignKey(
        ICD10Code, null=True, on_delete=models.SET_NULL)
    doctor_confirmed = models.BooleanField(default=False)
    ai_confidence = models.DecimalField(max_digits=5, decimal_places=2)

# Technology: BioBERT (medical NLP) OR Claude API
# Dataset: WHO ICD-10 full list (free download)
# pip install transformers torch # For BioBERT
```

4.4 React Native Mobile App

Screen	Patient Features	Doctor Features
Dashboard	Stats, quick actions, notifications bell	Today's schedule, stats
Appointments	Book, view, cancel, QR code	View, complete, write Rx
Lab Tests	Browse, book, download results	View patient results
Prescriptions	View, download PDF	Write, edit, view all
Payments	View history, pay online	N/A
Health Tools	Heart predictor, AI report reader	Patient reports viewer
Notifications	All notifications with links	All notifications
Profile	Edit, change password	Edit profile, schedule

```
# Technology stack for mobile:
npx create-expo-app hms-mobile --template blank-typescript
npm install @react-navigation/native axios
npm install react-native-paper # UI components
npm install @react-native-async-storage/async-storage # Token storage
npm install expo-notifications # Push notifications
npm install expo-document-picker # Upload reports to AI reader
```

4.5 Telemedicine — Video Consultations

Patients in remote areas can have video consultations with doctors. Especially valuable for follow-up appointments, prescription renewals, and specialist consultations without travel.

```
pip install agora-python-sdk # OR use Daily.co / Jitsi

# telemedicine/models.py

class VideoConsultation(models.Model):
    appointment = models.OneToOneField(
        'appointments.Appointment', on_delete=models.CASCADE)
    room_id = models.CharField(max_length=100, unique=True)
    room_token_patient = models.TextField() # Agora token
    room_token_doctor = models.TextField()
    scheduled_at = models.DateTimeField()
    started_at = models.DateTimeField(null=True)
    ended_at = models.DateTimeField(null=True)
    duration_minutes = models.IntegerField(null=True)
    status = models.CharField(
        max_length=15,
        choices=[('SCHEDULED', 'Scheduled'),
                 ('IN_PROGRESS', 'In Progress'),
                 ('COMPLETED', 'Completed'),
                 ('MISSED', 'Missed')]
    )
    recording_url = models.URLField(null=True, blank=True)
    notes = models.TextField(blank=True) # Doctor's notes during call
```

5.1 Docker + Docker Compose

```
# docker-compose.yml
version: '3.8'

services:
  web:
    build: .
    command: gunicorn hms_project.wsgi:application --bind 0.0.0.0:8000
    volumes:
      - ./app
    env_file:
      - .env
    depends_on:
      - db
      - redis

  db:
    image: postgres:15
    volumes:
      - postgres_data:/var/lib/postgresql/data/
    environment:
      POSTGRES_DB: hmsdb
      POSTGRES_USER: hmsuser
      POSTGRES_PASSWORD: ${DB_PASSWORD}

  redis:
    image: redis:7-alpine

  celery:
    build: .
    command: celery -A hms_project worker -l info
    depends_on:
      - redis
      - db

  celery_beat:
    build: .
    command: celery -A hms_project beat -l info

  nginx:
    image: nginx:alpine
    ports:
      - '80:80'
      - '443:443'
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
      - static_volume:/app/staticfiles

volumes:
  postgres_data:
```

```
static_volume:
```

5.2 CI/CD Pipeline (GitHub Actions)

```
# .github/workflows/deploy.yml
name: Deploy HMS

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Tests
        run: |
          pip install -r requirements.txt
          python manage.py test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Server
        run: |
          ssh user@server 'cd /var/www/hms && git pull && docker-compose up -d --build'
```

5.3 Monitoring & Alerting

Tool	Purpose	Free Tier?
Sentry	Error tracking — catch exceptions in production	Yes
Prometheus + Grafana	Performance metrics, response times, DB queries	Self-hosted
UptimeRobot	Uptime monitoring — alert if site goes down	Yes
Loguru	Structured logging to files and cloud	Yes
Django Silk	Database query profiling — find slow queries	Yes (dev only)

APPENDIX A — COMPLETE TECHNOLOGY STACK

Category	Technology	Version	Purpose
Core	Python	3.14+	Primary language
Core	Django	4.2+	Web framework
Core	PostgreSQL	15+	Production database
Cache	Redis	7+	Caching + Celery broker
Queue	Celery	5.3+	Async tasks
Queue	Celery Beat	5.3+	Scheduled tasks
API	Django REST Framework	3.14+	REST API
API	SimpleJWT	5+	JWT authentication
Real-time	Django Channels	4+	WebSockets
ML	scikit-learn	1.3+	Heart predictor, triage
ML	XGBoost	2+	No-show predictor
ML	Prophet	1.1+	Inventory forecasting
ML	Transformers	4+	BioBERT ICD coding
AI	Anthropic Claude API	Latest	Report reader, triage
NLP	RapidFuzz	3+	Drug name matching
PDF	ReportLab	4+	All PDF generation
QR	qrcode[pil]	7+	Appointment QR codes
Images	Pillow	10+	Image processing
SMS/WA	Twilio	Latest	WhatsApp + SMS
Payments	Khalti SDK	Latest	Nepal payments
Payments	Stripe	Latest	International payments
Security	django-axes	6+	Brute force protection
Security	django-otp	1.3+	Two-factor auth
Security	django-encrypted-model-fields	3+	Field encryption
Search	Elasticsearch	8+	Fast patient search
Video	Agora SDK	Latest	Telemedicine calls
Frontend	React + Vite	18+	Web SPA
Mobile	React Native + Expo	Latest	Mobile apps
Container	Docker + Compose	Latest	Containerization
Server	Nginx + Gunicorn	Latest	Production server
Monitoring	Sentry	Latest	Error tracking
CI/CD	GitHub Actions	Latest	Auto deployment

APPENDIX B — ALL DATABASE MODELS

App	Model	Key Fields
users	User	email, username, role, is_active, two_factor_enabled
users	PatientProfile	full_name, phone, blood_group, gender, dob, allergies[], search_vector
clinical	Specialization	name, description
clinical	Doctor	user, specialization, fee, experience, bio, photo
clinical	Shift	name, start_time, end_time, slot_duration
clinical	DoctorSchedule	doctor, day, shift
clinical	DoctorLeave	doctor, date, reason
appointments	Appointment	patient, doctor, date, start_time, end_time, status, qr_code, pdf_file
appointments	WaitingList	patient, doctor, date
prescriptions	Prescription	appointment, doctor, patient, notes, advice, follow_up_date, pdf_file
prescriptions	PrescriptionItem	prescription, medicine_name, dosage, frequency, duration, timing
payments	Payment	patient, doctor, appointment, lab_booking, admission, amount, status, type
payments	Refund	payment, reason, status, refunded_at
lab	TestTemplate	name, description, preparation, price, duration_minutes
lab	TestField	template, field_name, unit, normal_min, normal_max, is_required, order
lab	TestSchedule	template, day, start_time, end_time, max_bookings
lab	TestBooking	patient, template, date, status, payment_status, amount
lab	TestResult	booking, filled_by, notes, pdf_file, is_released
lab	TestResultItem	result, field, value, status(Normal/High/Low)
rooms	Room	room_number, room_type, price_per_day, capacity, floor, is_available
rooms	Admission	patient, doctor, room, appointment, admission_date, discharge_date, bill
notifications	Notification	user, type, title, message, is_read, appointment_id, lab_booking_id
audit	AuditLog	user, action, model_name, object_id, old_values, new_values, ip_address
drug_checker	DrugInteraction	drug_a, drug_b, severity, description, mechanism
drug_checker	DrugAllergyAlert	patient, drug_name, reaction, severity
triage	TriageAssessment	patient, priority, symptoms, vital_signs, ai_reasoning, confidence
triage	SymptomDatabase	name, keywords[], urgency_weight, red_flag
ml_noshow	NoShowPrediction	appointment, probability, risk_level, action_taken, actual_outcome
ml_noshow	NoShowMLModel	version, algorithm, accuracy, f1_score, model_file
vitals	VitalSigns	admission, temperature, bp, heart_rate, spo2, mews_score
vitals	VitalAlert	admission, alert_type, severity, mews_score, notified_doctor
ml_readmission	ReadmissionRisk	admission, risk_score, risk_factors, followup_scheduled
ml_heart	HeartRiskAssessment	patient, all 13 UCI features, risk_score, risk_level
inventory	Medicine	name, generic_name, current_stock, reorder_level, expiry_date
inventory	StockPrediction	medicine, predicted_usage_7d, predicted_stockout_date
ai_reader	ReportAnalysis	user, file, report_type, ai_summary, abnormal_findings, urgency
icd_coding	ICD10Code	code, description, category

icd_coding	DiagnosisCoding	prescription, raw_text, suggested_codes, selected_code
telemedicine	VideoConsultation	appointment, room_id, status, duration, recording_url
reviews	Review	patient, doctor, appointment, rating, comment

APPENDIX C — SECURITY CHECKLIST

✓ Authentication

- Email-based login (not username-based)
- Email activation required before login
- 2FA for Doctor, Admin, Lab Technician roles
- Session timeout after 30 minutes of inactivity
- Secure password requirements enforced
- Password reset via email token (expires in 1 hour)

✓ Access Control

- Role-based view access (@login_required + role check)
- Field-level security (receptionist cannot see diagnosis)
- Object-level security (patient can only see own data)
- API token required for all REST endpoints
- CORS configured (only approved domains)

✓ Data Security

- Sensitive fields encrypted at rest (diagnosis, lab values)
- HTTPS enforced in production (SSL certificate)
- Database credentials in environment variables only
- Media files not directly accessible (served via view)
- SQL injection prevention (Django ORM used throughout)
- XSS protection (Django template auto-escaping)
- CSRF protection enabled on all forms

✓ Monitoring

- Complete audit trail for every action
- Failed login attempts logged with IP
- Brute force lockout after 5 failed attempts
- Rate limiting on API endpoints
- Sentry error tracking in production
- Database backup every night
- Admin cannot delete audit logs

APPENDIX D — WHAT MAKES THIS UNIQUE

This is what separates your HMS from every other HMS project, tutorial, and open-source solution. Be able to explain each of these clearly.

◆ Predictive AI, Not Just Recording	Other HMS: Records what happened. Ours: Predicts what will happen. No-show predictor, readmission risk, vital sign deterioration — we prevent problems before they occur.
◆ Built for South Asia	WhatsApp-first communication (98% open rate vs 20% email). Nepal/India payment gateways (Khalti, eSewa). Multilingual support planned. Power-outage resistant offline mode.
◆ Drug Safety Built-In	The drug interaction checker is a clinical safety feature that literally saves lives. No other entry-level HMS has this. 200,000+ drug interactions checked automatically.
◆ AI Triage System	Patients are triaged by AI before seeing a doctor. Critical patients are flagged immediately. Reduces deaths from delayed treatment in busy hospitals.
◆ FHIR Compatible	HL7 FHIR API makes the system interoperable with government health systems, insurance companies, and international health data standards. Required for government hospital contracts.
◆ Real-Time Everything	WebSocket-powered live updates: vital sign alerts, triage queue, bed availability, doctor availability. Nurses see patient deterioration alerts instantly without refreshing.
◆ Partial Lab Results	Unique feature: Lab fields can be blank. Blank = shows as dash in report. Patient can get Fasting Blood Sugar only, without all other fields being required. Mirrors real hospital workflow.
◆ Complete Audit Trail	Every action logged with old and new values, IP address, and timestamp. Cannot be deleted. Full HIPAA-style compliance. Protects hospital from legal disputes.

ADVANCED HMS — Complete Project Roadmap

Generated by Claude AI | For Developer Reference Only | Build something that saves lives.